

1] ArrayBasics.java

Initially

marks = [0, 0, 0, 0, 0]

names = [null, null, null]

mark.length = 5

a = [10, 20, 30, 40, 0, 0, 0, 0] size = 4

$\therefore a[2] = 30$

insert 15 at 1st index -

size = 4, shift = 0

i = 4 $\rightarrow a[4] = a[3] \rightarrow 40$ shift 1

i = 3 $\rightarrow a[3] = a[2] \rightarrow 30$ shift 2

i = 2 $\rightarrow a[2] = a[1] \rightarrow 20$ shift 3

i = 1 $\rightarrow$ (condition false)

a[1] = 15

$\therefore a = [10, 15, 20, 30, 40, 0, 0, 0]$

size = 5, shift = 3

insert 50 at last [size = 5, index = 5] -

i = 5 $\rightarrow 5 > 5$ false

a[5] = 50

a = [10, 15, 20, 30, 40, 50, 0, 0] size = 6, shift = 0

Delete index 0 -

size = 6, index = 0

i[0] i = 0 $\rightarrow a[0] = a[1] = 15$

i = 1 $\rightarrow a[1] = a[2] = 20$

i = 2 $\rightarrow a[2] = a[3] = 30$

i = 3 $\rightarrow a[3] = a[4] = 40$

$i = 4 \rightarrow a[4] = a[5] \rightarrow 50$

$\therefore \text{shift} = 5 \quad a[5] = 0 \quad \text{size} = 5$

$a = [15, 20, 30, 40, 50, 0, 0, 0]$

if $a = 8 \rightarrow$ exception

(ArrayIndexOutOfBoundsException)

$x = 0 \rightarrow [1]$

$x = 1 \rightarrow [1, 1]$

$x = 2 \rightarrow [1, 2, 1]$

$x = 3 \rightarrow [1, 3, 3, 1]$

$a.length = 8, \quad a.length * 2 = 16$

bigger = $[15, 20, 30, 40, 50, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$

2.] StackArrayIntDemo.java

operation	stack	Top	o/p
initial	empty	-1	-
push 10	10	0	10
push 20	10, 20	1	20 10
peek	10, 20	1	20 10 Top = 20
display	10, 20	1	20, 10
push 30	10, 20, 30	2	30 20 10

operation	stack	Top	O/p
			40
push 40	10, 20, 30, 40	3	30
			20
			10
push 50	10, 20, 30, 40	3	overflow
pop	10, 20, 30	2	30
			20
			10
peek	10, 20, 30	2	Top = 30
exit	10, 20, 30	2	

3.] StackDemo_Generic<T>

• integer stack

max = 10 top = -1

push 10 : top = 0 & stack[0] = 10 20

push 20 : top = 1 & stack[1] = 20 10

pop (20) : top = 1 & stack[1] = 20

pop() : stack[1] = 20, top = 1
return 20

for string

max = 100 top = -1

push ("Hello") : top = 0

stack[0] = "Hello"

push ("World") : top = 1

stack[1] = "World"

World
Hello

pop() :

stack[i] = "World"

top = 0 return "World"

• isBalanced ("{[()] }")

expression = "{[()] }"

length = 6

max = 6 top = -1 stack = empty

i	ch	operation	Top	O/p
initial	-	-	-1	empty
0	{	push {	0	{
1	[	push [	1	{[
2	(	push (	2	{[(
3	)	pop) → match	1	{[
4	]	pop]	0	{
5	}	pop }	-1	empty

top = -1

isEmpty()

top == -1

-1 == -1 true

{[()] } = Balanced

4] StackDemo.java

max = 100

stack = char[100] top = -1

expression = "{ [()] }"

expression.length = 6

index	→	0	1	2	3	4	5	⋈
		{	[	(	)	]	}	

isBalanced()

top = -1 stack = empty

i = 0

ch = '{' push('{')

top = 0 ++top = 1

stack[i] = '{' top = 1

i = 0

ch = '{' push('{')

top = 1

++top = 0

stack[0] = '{'

top = 0

i = 1

ch = '[' push('[')

top = 0

++top = 1

stack[1] = '['

top = 1

i = 2

ch = 'c'

push('c')

top = 1

++top = 2

stack[2] = 'c'

top = 2

i = 3

ch = ']'

isEmpty() : top == -1
2 == -1 → false

open = pop()

open = stack[2] = '['

top -- → 1

check → isMatchingPair('[' , ']') → true

i = 4

ch = ']'

isEmpty() : 1 == -1 → false

open = pop()

open = stack[1] → '['

top -- → 0

isMatchingPair('[' , ']') → true

i = 5

ch = '}'

isEmpty() : 0 == -1 → false

open = pop()

open = '{'

top -- → -1

isMatchingPair('{', '}') → true

top == -1

-1 == -1 → true

∴ isBalanced("{ [()] }") → true

5] ExpressionEvaluationDemo.java

input = 2 3 1 * + 9 -
expression.trim().split()

[2, 3, 1, *, +, 9, -] stack = empty

steps	Input	operation	stack
			empty
1	2	push 2	[2]
2	3	push 3	[2, 3]
3	1	push 1	[2, 3, 1]
4	*	pop 1, pop 3 $\rightarrow 3 * 1 = 3$ push 3	[2, 3]
5	+	pop 3, pop 2 $\rightarrow 2 + 3 = 5$ push 5	[5]
6	9	push 9	[5, 9]
7	-	pop 9, pop 5 $\rightarrow 5 - 9 = -4$ push -4	[-4]

input = 2	not operator	push(2)	1
input = 3	not operator	push(3)	3
input = 1	not operator	push(1)	2
input = *	pop 1 & 3 $1 * 3 = 3$	push(3)	3
			2
input = +	pop 3 & 2 $2 + 3 = 5$ push(5)		5

input = 9 push(9)

9
5

input = - pop 9 & 5

$$5 - 9 = -4$$

push(-4)

-4

6.] ParenthesisMatchDemo.java

initially stack = empty

• (a+b) * (c+d)

(→ push → [(]

) → pop (→ match → []

(→ push → [(]

) → pop (→ match → []

final stack = empty

result = Balanced

• {[()]}

{ → push → [{]

[→ push → [{, []

(→ push → [{, [, (]

) → pop (→ match → [{, []

] → pop [→ match → [{]

} → pop { → match → []

Balanced

• (a+b)

• (a+b] * c)

c → push → [(]

] → pop (→ not matching return false
not Balanced

• $(x+y)$

$(\rightarrow \text{push} \rightarrow [c]$

$(\rightarrow \text{push} \rightarrow [c, c]$

$) \rightarrow \text{pop}) \rightarrow \text{match} \rightarrow [c]$
loop end

stack $[c]$ isEmpty() = false
not Balanced

7.] ReverseStringDemo.java

input = "Hello"

length = 5

stack = empty

• input.charAt(i)

i = 0	push(H)	[H]	O
i = 1	push(E)	[H, E]	L
i = 2	push(L)	[H, E, L]	L
i = 3	push(L)	[H, E, L, L]	E
i = 4	push(o)	[H, E, L, L, O]	H

pop() = "o"	[H, E, L, L]	output O
pop() = "L"	[H, E, L]	OL
pop() = "L"	[H, E]	OLL
pop() = "E"	[H]	OLLE
pop() = "H"	[]	OLLEH

8] StackUsingLinkedListDemo.java

top = null size = 0 stack = empty

push 10

newNode.data = 10

newNode.next = null

top = newNode

size = 1

stack: Top → [10 | null]

push 20

newNode.data = 20

newNode.next = top

size = 2

stack: Top [20 |] → [10 | null]

peek

top != null → true

top.data = 20

stack unchanged

Display

current.data = 20

current = current.next

current.data = 10

current = null

o/p 20 10

size = 2

pop

top != null $\rightarrow$ true

value = top.data = 20

size -- ; $\therefore$ size = 1

9.] stack Using LinkedList.java

operations	stack	Top	size of stack
initially	empty	null	0
push 10	10	10	1
push 20	20, 10	20	2
push 30	30, 20, 10	30	3
display	30, 20, 10	30	3
peek	30, 20, 10	30	3
pop	20, 10	20	2
size	20, 10	20	2
display	20, 10	20	2

10.] UndoOperationDemo.java

choice	operation	stack	output
	initial	empty	
1	add open file	open file	saved
2	add edit file	open file, edit file	saved
1	add save file	open file, edit file, save file	saved
2	undo	open file, edit file	save undo
3	show	open file, edit file	display

Queue :

1] QueueUsingStackDemo.java

Initially

stackNewest = []

stackOldest = []

top = -1

top = -1

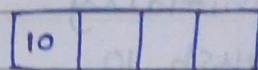
Queue = empty

• enqueue 10

stackNewest.push(10)

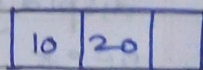
top = 0

Queue = 10

↑
enqueue←
enqueue

• enqueue 20

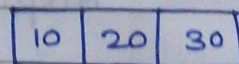
top = 1 stackNewest = [10, 20]



Queue = 10, 20

• enqueue 30

top = 2 stackNewest = [10, 20, 30]



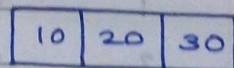
Queue = 10, 20, 30

• peek

stackOldest = []

stackOldest = [30, 20, 10]

Top



top = 2 output = 10

o/p = 10

• display (pop from stackOldest)

o/p → 10, 20, 30

• Dequeue

stackOldest = [30, 20]

size = 0

2] queueDemo.java

max = 5

queue []

queue = empty

0	1	2	3	4
---	---	---	---	---

front = -1 rear = -1

• enqueue (10)

rear = max - 1

 $-1 = 4$ false

front == -1 true

front = 0

++rear

 $\therefore$ rear = 0 front = 0

10			
----	--	--	--

↑
front
↑
rear

• enqueue (20)

front = 0

rear = 0 + 1 = 1

10	20		
----	----	--	--

↑ ↑
front rear

• enqueue (30)

front = 0

rear = 2

10	20	30	
----	----	----	--

↑ ↑
front rear

• dequeue

front = 1

rear = 2

10	20	30	
----	----	----	--

↑ ↑
front rear

• dequeue

front = 2

rear = 2

10	20	30
----	----	----

↑
front / rear